Bird Species Identification using Deep Learning

UE24CS352A – Machine Learning · Mini-Project Write-up
Bhavyaa Garg (PES1UG24CS116) and Bhavesh Velluru (PES1UG24CS115) · Section B, CSE · PES University

1. Problem statement

Given a photograph of a bird, identify its species. This is a supervised, fine-grained, 200-class image-classification problem: the classes are visually very similar and the images vary in pose, lighting and background. Our target was at least 85% accuracy on the official test split. We compare four models, from a small CNN trained from scratch to a frozen DINOv2 feature extractor with a logistic-regression classifier, and wrap the best one in a web demo.

2. Dataset

We use the original Caltech-UCSD Birds-200-2011 (CUB-200-2011) dataset: 11,788 images of 200 bird species. We keep the official split of 5,994 training and 5,794 test images. A validation set of 900 images (15%, stratified by class, seed 42) is carved from the official training split, leaving 5,094 images for training. All model selection (epochs, learning rates, regularisation) used the validation split only; the test split was used once, for the final evaluation.

3. Methodology

- **1. CNN baseline:** TensorFlow, from scratch: three conv layers (32/64/128 filters, ReLU, max-pooling), global average pooling, dropout 0.3, 200-way softmax. Input 224×224, random flip and brightness augmentation, Adam lr 1e-3, batch 32, sparse categorical cross-entropy, 12 epochs, early stopping on validation accuracy (patience 3).
- **2. MobileNetV2 transfer:** ImageNet-pretrained backbone frozen, global average pooling, dropout 0.25, softmax head; Adam lr 1e-4, batch 32, 8 epochs. Validation accuracy was still rising when it stopped (underfitting).
- **3. MobileNetV2 fine-tuned:** stage 1 trains the head at lr 1e-3 with extra augmentation (zoom 0.15, rotation 0.05, translation 0.1, contrast 0.15; stopped after 15 of 20 epochs, best at epoch 12); stage 2 unfreezes the top 40 backbone layers (BatchNorm kept frozen) at lr 1e-5 (stopped after 8 of 10 epochs, best at epoch 5).
- **4. DINOv2 + logistic regression (best):** frozen DINOv2 ViT-B/14 (self-supervised, pretrained by Meta on LVD-142M; timm vit_base_patch14_dinov2.lvd142m) gives 768-d features from a 224×224 centre crop with ImageNet normalisation, averaged over the image and its horizontal flip. A standardised multinomial logistic regression (lbfgs, max 3000 iterations) classifies them. The regularisation strength C was chosen on validation from {0.01, 0.03, 0.1, 0.3} (88.56%, 88.33%, 88.33%, 88.22%); C = 0.01 was selected.

Models 1–3 use TensorFlow/Keras; model 4 uses PyTorch (timm) and scikit-learn. Hardware: Apple M2 laptop, 8 GB RAM; TensorFlow ran on the CPU and PyTorch on the M2 GPU (MPS). DINOv2 feature extraction for all 11,788 images took about 40 minutes.

4. Implementation

- **src/data.py**: reads CUB metadata, builds splits and TensorFlow datasets
- **src/features.py**: loads DINOv2 and extracts features (GPU when available)
- **train.py**: trains the baseline, transfer and fine-tuned TensorFlow models
- **train_features.py**: extracts/caches DINOv2 features, picks C on validation, saves the classifier, evaluates on test
- **evaluate.py**: evaluates saved TensorFlow models; writes metrics, classification report and confusion matrix
- **app.py**: Streamlit demo using the DINOv2 model: upload a photo, see the predicted species and top-five scores

Commands: *python train.py --model baseline --epochs 12 · python train.py --model transfer --epochs 8 · python train.py --model finetune --epochs 20 --finetune-epochs 10 · python train_features.py --backbone dinov2_base · python evaluate.py --model models/<name>.keras · streamlit run app.py*

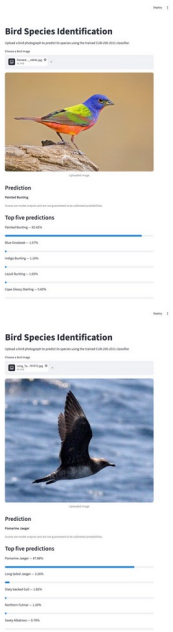


Fig. 1: Streamlit demo, correct (left) and failure case (right)

5. Results (official test split, 5,794 images)

Model	Train acc.	Val. acc.	Test top-1	Test top-5	Macro F1
CNN baseline	2.9%	3.8%	3.2%	13.0%	0.018
MobileNetV2 transfer	51.6%	44.9%	45.8%	77.2%	0.444
MobileNetV2 fine-tuned	90.5%	57.4%	57.5%	85.8%	0.579
DINOv2 ViT-B/14 + logistic regression	**98.2%**	**88.6%**	**89.8%**	**98.7%**	**0.897**

Chance level is 0.5%. Keras training accuracy is the value logged at the best-validation epoch (augmentation and dropout active); for DINOv2 it is measured on un-augmented training features. Only the DINOv2 model meets the 85% target.

The baseline barely beats chance: about 25 training images per species is far too few to learn fine-grained features from scratch. Frozen MobileNetV2 reached 45.8% but underfitted. Fine-tuning (higher head learning rate, longer training, stronger augmentation, partial unfreezing) raised test accuracy to 57.5%, but then overfitted: 90.5% train vs 57.5% test, a 33-point gap, and unfreezing added only about 2 points. Better regularisation of MobileNetV2 alone could not reach 85%; the limit was the quality of the pretrained features. Frozen DINOv2 features with a regularised linear classifier gave 88.6% validation and 89.8% test, cutting the gap to about 8 points. Validation and test agree closely for every model, so the evaluation looks reliable.

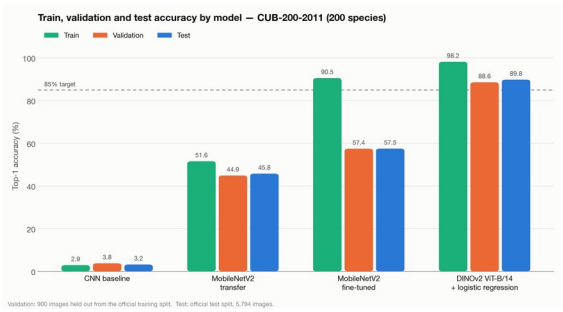


Fig. 2: Train, validation and test accuracy per model, with the 85% target.

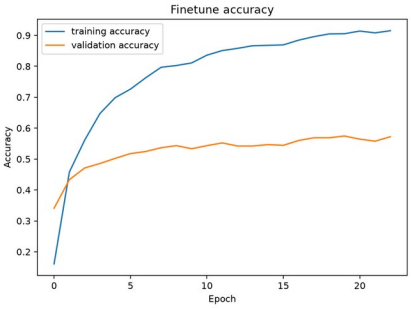


Fig. 3: Fine-tuned MobileNetV2, training vs validation accuracy (overfitting).

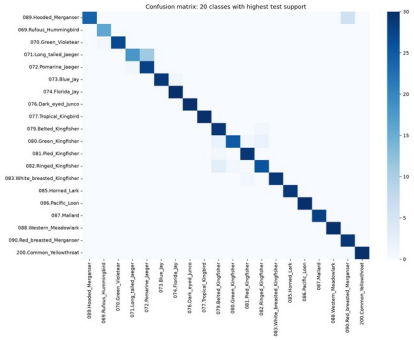


Fig. 4: DINOv2 confusion matrix, 20 species with the most test images (rows true, columns predicted).

Error analysis (DINOv2). The main confusions are between visually similar species: Long-tailed Jaeger is often predicted as Pomarine Jaeger, Hooded Merganser sometimes as Red-breasted Merganser, and Green and Ringed Kingfishers are confused with other kingfishers. In the demo, a Cedar Waxwing is predicted correctly at 78.6% with the very similar Bohemian Waxwing second at 14.9%. Failure case (Fig. 1, right): a blurry in-flight photo of a Long-tailed Jaeger is predicted as Pomarine Jaeger at 87.88%, with Long-tailed Jaeger second at 3.26%. Prediction takes under 0.1 s per image once the model is loaded; scores are model outputs, not calibrated probabilities.

6. Conclusion and limitations

The best model is frozen DINOv2 ViT-B/14 with logistic regression: 89.8% test top-1, 98.7% top-5 and macro F1 0.897, about 32 points above fine-tuned MobileNetV2 and the only model to meet the 85% target. Limitations: (1) the pretraining data (ImageNet for MobileNetV2, LVD-142M for DINOv2) may overlap CUB images, so accuracy may be somewhat optimistic; (2) DINOv2 is used as a frozen feature extractor, so the learning here is the classifier and the model comparison, not end-to-end deep-network training; (3) each result is a single run with seed 42, with no variance measured; (4) the project mixes two frameworks (TensorFlow and PyTorch); (5) look-alike species remain the main source of errors. Next steps: fine-tune DINOv2 itself, crop to the bird using CUB's bounding boxes (the demo would then need a bird detector), and try the larger DINOv2 ViT-L/14.

7. Individual contributions

The work was divided between the two team members as set out below.

Bhavesh (CS115) set up the CUB-200-2011 dataset and built the data pipeline, including the official train/test split and the stratified validation split (src/data.py). He implemented and trained the CNN baseline (train.py). He also analysed the training curves and the overfitting of the fine-tuned model. He wrote the Dataset, Methodology (TensorFlow models) and Implementation sections.

Bhavyaa (CS116) implemented the DINOv2 feature-extraction pipeline and the logistic-regression classifier, including selecting the regularisation strength on the validation split (src/features.py, train_features.py). She wrote the evaluation code (evaluate.py), produced the metrics and confusion matrix, carried out the error and failure-case analysis, and built the Streamlit demo (app.py). She wrote the Results, Error analysis and Methodology (DINOv2) sections.

Both members worked together on the MobileNetV2 transfer-learning and two-stage fine-tuning models (train.py), jointly compared the models, wrote the introduction, conclusion and limitations, prepared the presentation, and maintained the project repository.

References

Wah, C. et al. The Caltech-UCSD Birds-200-2011 Dataset. California Institute of Technology, 2011. · Sandler, M. et al. MobileNetV2: Inverted Residuals and Linear Bottlenecks. CVPR, 2018. · Oquab, M. et al. DINOv2: Learning Robust Visual Features without Supervision. TMLR, 2024. · TensorFlow/Keras documentation, tensorflow.org. · PyTorch, pytorch.org. · PyTorch Image Models (timm), github.com/huggingface/pytorch-image-models.